PUC Minas



Aula 02 · Deploy

Docker: imagens, containers e comandos essenciais

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Diferenciar **imagem**, **container**, **Dockerfile** e **registry**

2

Instalar o **Docker Desktop** e testar com `hello-world`

3

Rodar containers com portas (`-p`), variáveis (`-e`) e nomes

4

Ver logs, entrar no container e limpar recursos

5

Entender **camadas** e **cache** de build

Apresentação da aula

Antes de começar

- Aula 01 desta pasta

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O problema que o Docker resolve

02 Os conceitos

03 Instalação

04 Rodando containers

05 Comandos do dia a dia

06 Camadas e cache

07 Aprofundando: Docker no dia a dia

01

O problema que o Docker resolve

O problema que o Docker resolve

“Na minha máquina funciona.” O seu Spring Boot precisa do JDK 25; o do colega usa 21; o servidor tem 17. O Docker empacota a aplicação **junto com tudo que ela precisa** (sistema base, JDK, arquivos) em uma **imagem**. Essa imagem roda igual em qualquer lugar que tenha Docker: no seu notebook, no do professor e no Render.

01

O problema que o Docker resolve

Aula 02 · Deploy

02

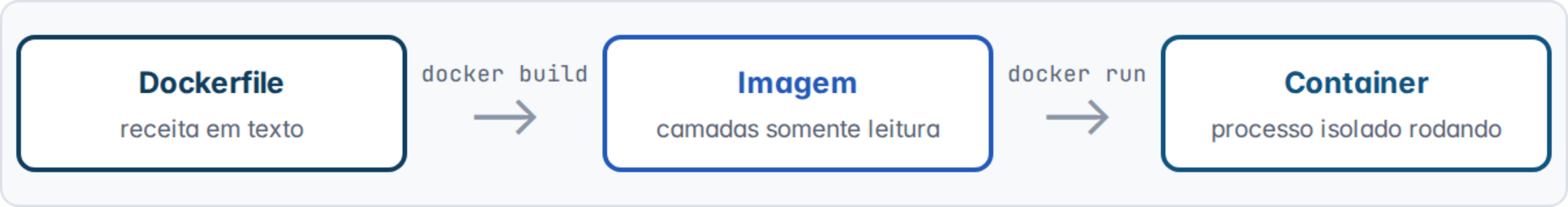
Os conceitos

Os conceitos

Conceito	Analogia	Exemplo
Dockerfile	A receita	Arquivo de texto com os passos para montar a imagem
Imagem	O bolo pronto, congelado	`eclipse-temurin:25-jre`, `meu-back:1.0`
Container	Uma fatia servida	Uma imagem **em execução**, isolada, com sua própria rede e arquivos
Registry	A padaria	**Docker Hub**: de onde vêm imagens prontas como `maven`, `node`, `nginx`, `mongo`
Tag	A versão	`node:22-alpine`, `maven:3.9-eclipse-temurin-25`

Os conceitos

Container **não é** máquina virtual: ele compartilha o núcleo (*kernel*) do sistema e sobe em segundos, usando bem menos memória.



Do arquivo ao processo.

03

Instalação

Instalação

- **Windows e macOS**: instale o **Docker Desktop** (docker.com). No Windows ele usa o **WSL 2**; aceite a instalação quando for pedida e reinicie.
- **Linux**: instale o *Docker Engine* pelo repositório oficial e adicione seu usuário ao grupo `docker` (`sudo usermod -aG docker $USER`, depois saia e entre de novo).

03

Instalação

Aula 02 · Deploy

Instalação

Terminal

```
aluno@pc:~$ docker --version
Docker version 28.5.1, build e180ab8
aluno@pc:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Teste da instalação: o Docker baixa a imagem do Docker Hub e roda um container.

Instalação

Erro comum

`Cannot connect to the Docker daemon` / `docker: error during connect`: o Docker Desktop não está aberto (Windows/macOS) ou o serviço não está ativo (Linux: `sudo systemctl start docker`). `permission denied ... docker.sock` no Linux: falta o grupo `docker`.

04

Rodando containers

Rodando containers

Vamos subir um MongoDB local, sem instalar nada além do Docker. É o mesmo serviço `mongo` do `docker-compose.yml` do `PalpiteDTI`:

Terminal

```
aluno@pc:~$ docker run -d --name mongo-aula -p 27017:27017 mongo:7
Unable to find image 'mongo:7' locally
7: Pulling from library/mongo
Status: Downloaded newer image for mongo:7
a3f9c1e2b7d4...
aluno@pc:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  STATUS         PORTS                     NAMES
a3f9c1e2b7d4   mongo:7   "docker-entrypoint.s…"   Up 8 seconds   0.0.0.0:27017→27017/tcp   mongo-aula
```

MongoDB rodando em container. O Spring com `mongodb://localhost:27017` já consegue conectar.

Rodando containers

Opção do `docker run`	Significado
`-d`	*Detached*: roda em segundo plano
`--name mongo-aula`	Dá um nome (senão o Docker inventa um)
`-p 27017:27017`	**porta do seu computador : porta do container**
`-e CHAVE=valor`	Define uma variável de ambiente dentro do container
`--env-file .env`	Lê várias variáveis de um arquivo
`-v dados:/data/db`	Volume: guarda os dados fora do container (sobrevivem ao `rm`)
`--rm`	Apaga o container quando ele parar
`-it`	Modo interativo com terminal

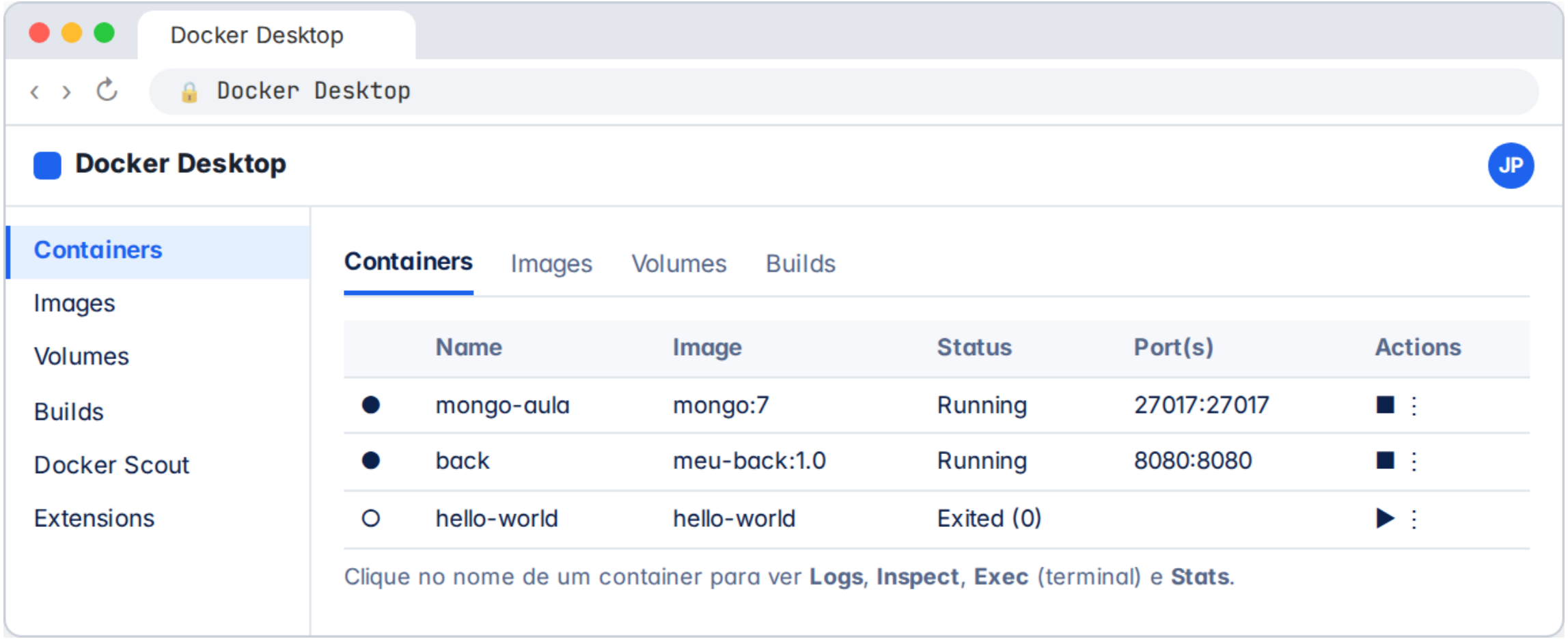
05

Comandos do dia a dia

Comandos do dia a dia

Comando	O que faz
`docker ps` / `docker ps -a`	Containers rodando / todos, inclusive parados
`docker images`	Imagens baixadas ou construídas
`docker logs -f mongo-aula`	Mostra (e acompanha) a saída do container. **O mais usado para depurar.**
`docker stop mongo-aula` / `docker start mongo-aula`	Para / inicia de novo
`docker rm mongo-aula`	Remove o container (parado)
`docker exec -it mongo-aula bash`	Abre um terminal **dentro** do container
`docker build -t meu-back .`	Constrói uma imagem a partir do Dockerfile da pasta atual
`docker system prune`	Limpa containers parados, redes e cache sem uso

Comandos do dia a dia



O Docker Desktop mostra os mesmos containers do `docker ps`, com botões para parar, iniciar e ver logs.

06

Camadas e cache

Camadas e cache

Cada instrução do Dockerfile gera uma **camada**. Ao reconstruir, o Docker reaproveita as camadas cujas instruções e arquivos **não mudaram**. Por isso os Dockerfiles dos projetos copiam primeiro só o `pom.xml` (ou `package.json`), baixam as dependências e **só depois** copiam o código: ao mudar uma linha de Java, o passo demorado de baixar dependências vem do cache.

06

Camadas e cache

Aula 02 · Deploy

Camadas e cache

Terminal

```
aluno@pc:~/projeto/Back$ docker build -t meu-back .
[+] Building 12.4s (14/14) FINISHED
 ⇒ [build 2/6] WORKDIR /app                                  0.0s
 ⇒ CACHED [build 3/6] COPY pom.xml .                         0.0s
 ⇒ CACHED [build 4/6] RUN mvn -q -B dependency:go-offline    0.0s
 ⇒ [build 5/6] COPY src ./src                                0.1s
 ⇒ [build 6/6] RUN mvn -q -B clean package -DskipTests      10.8s
 ⇒ [stage-1 3/3] COPY --from=build /app/target/*.jar app.jar 0.2s
```

Segundo build após mudar só código Java: as dependências vêm do cache (`CACHED`).

07

Aprofundando: Docker no dia a dia

Aprofundando: Docker no dia a dia

Instrução do Dockerfile	Para que serve
`FROM`	Imagem base de cada etapa
`WORKDIR`	Pasta de trabalho
`COPY` / `ADD`	Copiar arquivos para a imagem (`ADD` também extrai `.tar` e baixa URLs)
`RUN`	Executar um comando no build
`ENV`	Variável de ambiente disponível no build e na execução
`ARG`	Valor disponível só no build (`--build-arg`)
`EXPOSE`	Documenta a porta
`ENTRYPOINT` / `CMD`	Comando principal / argumentos padrão

Volume x bind mount

Volume (`-v dados:/data/db`)

- Gerenciado pelo Docker
- Ideal para dados de banco
- Sobrevive ao `docker rm`

vs

Bind mount (`-v ./pasta:/app`)

- Liga uma pasta do seu computador
- Ideal para arquivos de configuração
- Mudanças aparecem dos dois lados na hora

Aprofundando: Docker no dia a dia

Banco relacional local em um comando

```
# PostgreSQL local, para testar o JPA sem o Supabase
docker run -d --name pg-aula -p 5432:5432 \
  -e POSTGRES_PASSWORD=aula -e POSTGRES_DB=livraria \
  -v pg-dados:/var/lib/postgresql/data postgres:17

# application.properties local
# spring.datasource.url=jdbc:postgresql://localhost:5432/livraria
# spring.datasource.username=postgres
# spring.datasource.password=aula
```

Aprofundando: Docker no dia a dia

Entrando no container do MongoDB

```
$ docker exec -it mongo-aula mongosh
test> show dbs
admin   40.00 KiB
exemplo_integracao  72.00 KiB
test> use exemplo_integracao
exemplo_integracao> db.alunos.find().limit(1)
[ { _id: ObjectId('6714c0f2a1b3c45d6e7f8091'), nome: 'Ana Souza', ... } ]
```

Aprofundando: Docker no dia a dia

Problema	Comando para investigar
Container parou logo depois de subir	`docker logs`
Porta já em uso	`docker ps` e pare o container que usa a porta
Disco cheio de imagens antigas	`docker system df` e `docker system prune`
Quer ver variáveis e rede	`docker inspect`
Consumo de CPU/memória	`docker stats`

Boas práticas

Imagens oficiais

Use imagens oficiais do Docker Hub (`eclipse-temurin`, `node`, `nginx`, `mongo`).

Tags fixas

`node:22-alpine` em vez de `node:latest`: o build não muda sozinho.

Imagens pequenas

Multi-stage e variantes `-alpine`/`-jre` sobem mais rápido no Render.

Sem segredos na imagem

Senhas entram como variáveis (`-e`) na execução, nunca no Dockerfile.

Aprofundando: Docker no dia a dia

Observação

No Windows, o Docker Desktop roda os containers dentro do **WSL 2**. Se o build estiver muito lento, guarde o projeto dentro do sistema de arquivos do Linux (`\\wsl$\Ubuntu\home\...`) em vez de `C:\Users\...`.

08

Quiz de revisão

Pergunta

Qual a diferença entre imagem e container?

A. São a mesma coisa

B. Imagem é o pacote; container é a imagem em execução

C. Container é o Dockerfile

D. Imagem é o banco

Resposta

Qual a diferença entre imagem e container?

A. São a mesma coisa

✓ Imagem é o pacote; container é a imagem em execução

C. Container é o Dockerfile

D. Imagem é o banco

Por quê: Um Dockerfile gera a imagem com docker build.

Pergunta

Em `-p 8081:8080`, qual é a porta do seu computador?

A 8080

B 8081

C As duas

D Nenhuma

Quiz · questão 2 de 5

Resposta

Em `-p 8081:8080`, qual é a porta do seu computador?

A 8080

✓ 8081

C As duas

D Nenhuma

Por quê: Formato host:container.

Pergunta

Qual comando mostra a saída de um container?

A docker ps

B docker logs -f nome

C docker images

D docker build

Resposta

Qual comando mostra a saída de um container?

A docker ps

✓ docker logs -f nome

C docker images

D docker build

Por quê: É o mais usado para depurar.

Pergunta

Por que copiar o pom.xml antes do src no Dockerfile?

A Exigência do Maven

B Aproveitar o cache da camada de dependências

C Por segurança

D Para o CORS

Resposta

Por que copiar o pom.xml antes do src no Dockerfile?

A Exigência do Maven

✓ Aproveitar o cache da camada de dependências

C Por segurança

D Para o CORS

Por quê: Mudou só o código? As dependências vêm do cache.

Pergunta

Para que serve um volume no MongoDB em container?

A. Acelerar consultas

B. Manter os dados depois de remover o container

C. Liberar a porta

D. Criar usuários

Resposta

Para que serve um volume no MongoDB em container?

A Acelerar consultas

✓ Manter os dados depois de remover o container

C Liberar a porta

D Criar usuários

Por quê: -v mongo-dados:/data/db.

Fechamento

Resumo da aula

- ✓ **Dockerfile** (receita) → **imagem** (pacote) → **container** (em execução); imagens vêm do **Docker Hub**.
- ✓ `docker run -d --name x -p host:container -e VAR=v imagem`.
- ✓ `docker ps`, `logs -f`, `exec -it`, `stop`, `rm`, `images`, `build -t`.
- ✓ Camadas são cacheadas: copie primeiro o arquivo de dependências, depois o código.
- ✓ O Render constrói e roda a sua imagem exatamente desse jeito.

Exercícios

1. Instale o Docker e rode `hello-world`.

2. Suba um `mongo:7` e conecte o projeto `Exemplo_Integracao/Back` nele.

3. Suba um `postgres:17` com `-e POSTGRES_PASSWORD=aula` e conecte um projeto JPA.

4. Use `docker exec` para abrir o `mongosh` dentro do container e liste os bancos.

Para ir além

- docs.docker.com/get-started
- hub.docker.com

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 02?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web